

Executable Diagnostic Coordinates: A Reference Implementation of the Repair-Warrant Objective

Flyxion
Independent Researcher

August 2026

Abstract

Diagnostic Coordinates and the Time of Repair specifies a repair-warrant objective, $J(a \mid h) = E(a \mid h) + \lambda R(a \mid h) + \mu \Delta_{\text{dep}}(a \mid h, R(h)) - \nu M(a \mid h)$, together with a three-way refusal and escalation taxonomy governing when an intervention is warranted, when no intervention beats refusal, when the governing constraint structure supplies no admissible comparison class at all, and when it cannot yet be determined which of those two failures has occurred. This note describes a small Rust reference implementation of that apparatus and argues that the exercise of implementing it, rather than merely restating it, surfaces two things the prose specification leaves underdetermined: a degenerate case in the structural-distance term when the reference class is empty, and a sensitivity of the taxonomy's outcome to the relative weighting of the objective's terms that is easy to state abstractly but easy to get backward in practice. The implementation is offered as a scaffold for testing specific instantiations of E , R , and M against the theory, not as a claim that any particular instantiation is correct.

1 Motivation

A formal specification can be checked for internal consistency by inspection: does the Repair-Warrant Criterion follow from the definitions, do the three taxonomy branches partition the space of outcomes, is the null intervention handled correctly. What inspection cannot check is whether the specification, once instantiated with concrete scoring functions and run against concrete cases, behaves the way its prose suggests it will. The gap between a specification that is internally consistent and an instantiation that behaves as intended is exactly the gap an executable reference implementation is suited to closing, because it forces every term in the objective to be given a computable meaning and then forces that meaning to be run against cases designed to exercise each branch of the taxonomy separately.

This note documents such an implementation, written in Rust and organized as a small library crate of five modules together with a worked example and an accompanying test suite. The implementation is deliberately thin. It does not take a position on what evidence, regularization cost, or marginal value mean for any particular theory being repaired; those three terms are left as a trait the caller must implement. What is fixed is the algebraic shape of the objective, the three-way branch structure at the point where the reference class is

derived, and the Repair-Warrant Criterion’s requirement that the null intervention evaluate to exactly zero. The remainder of this note recapitulates the theoretical apparatus briefly, describes the implementation’s structure, and then reports two findings from the process of getting the accompanying test suite to pass, each of which is a small but genuine piece of information about the objective’s behavior rather than an artifact of the code.

2 The objective and the taxonomy, recapitulated

A history h carries repair-time boundary data $B(h)$, a set of facts the intervention is not entitled to rewrite to the extent those facts are classified irreversible. A governing constraint structure Γ_h , whose provenance must be independent of h itself, generates a reference class of admissible continuations $R_\Gamma(h) = \text{Ext}_{\Gamma_h}(B(h))$. An intervention a acting on h produces a post-intervention history h' , and the objective

$$J(a \mid h) = E(a \mid h) + \lambda R(a \mid h) + \mu \Delta_{\text{dep}}(a \mid h, R(h)) - \nu M(a \mid h)$$

scores that intervention against evidence E , a regularization term R , the marginal structural effect Δ_{dep} , and a marginal-value term M , with non-negative weights λ, μ, ν . The marginal structural effect is defined through a structure-sensitive distance $\delta_{\text{dep}}(h, R(h))$ from a history to the reference class, soft-min aggregated over the class, and is itself the difference $\delta_{\text{dep}}(h', R(h)) - \delta_{\text{dep}}(h, R(h))$: negative when the intervention moves the history structurally closer to an admissible continuation, positive when it moves the history further away. The Repair-Warrant Criterion states that the null intervention satisfies $J(\emptyset \mid h) = 0$ identically, and that intervention is warranted only if some non-null a achieves $J(a \mid h) < 0$.

Deriving the reference class can fail in two structurally distinct ways, and the taxonomy that follows from that distinction replaces what was originally a single overloaded failure condition. If every fact in $B(h)$ has been classified, either as irreversible or as revisable, and $\text{Ext}_{\Gamma_h}(B(h))$ is still empty, the emptiness is a genuine finding: the present repair frame, meaning the pairing of the boundary data and the constraint structure, supplies no admissible comparison class at all. This is branch two, and it forces repair to become horizontal, revising either the constraint structure, the vocabulary it is expressed in, or the classification of some boundary fact previously taken to be irreversible. If instead some fact in $B(h)$ has not yet been classified, no verdict about $\text{Ext}_{\Gamma_h}(B(h))$, empty or otherwise, can be trusted, because it is impossible to say whether an eventual obstruction belongs to the boundary data or to the constraint structure. This is branch three, a diagnostic-coordinate deficiency distinct from either component actually being at fault, and its resolution is classification of the offending fact rather than revision of Γ_h . Only once the reference class is well-formed does branch one become the relevant question: whether some non-null intervention achieves $J(a \mid h) < 0$, or whether, as is the ordinary case, no intervention beats refusal.

3 Implementation structure

The crate is organized into five modules, each corresponding to one piece of the apparatus above. The domain module defines a history as a feature vector together with an identi-

fier, a continuation with the same shape, an intervention carrying an effect vector with the null intervention distinguished by convention, and the boundary data as a list of facts each classified irreversible, revisable, or unclassified. Applying an intervention to a history is implemented as elementwise addition of the effect vector, padded to matching length; this is the one place where the implementation commits to a concrete, and admittedly minimal, notion of what an intervention does to a history, and it is the first thing to replace when testing a specific theory rather than the scaffolding itself.

The reference-class module represents Γ_h as a named generator function from boundary data to a list of continuations, deliberately leaving that function’s internals to the caller, since what a constraint structure actually is (a grammar, a type system, a set of physical laws) is theory-specific in a way nothing in this crate should presume to settle. The module’s substantive content is the three-way split at $\text{Ext}_\Gamma(B(h))$: a non-empty outcome carrying the reference class, an empty-determined outcome reached only once every boundary fact has been classified, and an undetermined outcome reached as soon as any fact has not been. This split is checked before the generator is even consulted when an unclassified fact is present, which is the implementation’s way of enforcing, structurally rather than by convention, that branch three take precedence over branch two: an unclassified fact makes any verdict about emptiness untrustworthy regardless of what the generator would have returned.

```
pub enum ExtOutcome {
    NonEmpty(Vec<Continuation>),
    EmptyDetermined,
    Undetermined,
}

pub fn ext(gamma: &ConstraintStructure, boundary: &BoundaryData) -> ExtOutcome
{
    if boundary.has_unclassified() {
        return ExtOutcome::Undetermined;
    }
    let continuations = (gamma.generator)(boundary);
    if continuations.is_empty() {
        ExtOutcome::EmptyDetermined
    } else {
        ExtOutcome::NonEmpty(continuations)
    }
}
```

The objective module implements J itself, together with δ_{dep} as a log-sum-exp soft-minimum over Euclidean feature distances to the reference class, sharpened or softened by a temperature parameter, and Δ_{dep} as the difference of that quantity before and after applying the intervention. E , R , and M are exposed as a trait, `ObjectiveTerms`, that any caller implements against their own notion of evidence, regularization, and marginal value; the crate supplies no default for any of the three, on the view that supplying one would misrepresent how settled those notions currently are.

The taxonomy module is a direct transcription of the three branches described above into an enumeration, and the ledger module, `RepairLedger`, is the piece that ties the oth-

ers together into a single decision procedure: given a history, boundary data, and a list of candidate interventions, it derives the reference class, routes to branch two or three if that derivation does not yield a usable class, and otherwise evaluates every candidate against J and returns either the best warranted intervention or a branch-one refusal carrying the best (non-warranting) candidate found. The ledger’s `StopReason` type is written to subsume, rather than replace, the three-variant `StopReason` already in use in an existing Rust implementation (entheai’s `repair_stop.rs`), whose `MaxAttempts`, `NoMarginalValue`, and `WeakVerifier` variants are all, on inspection, different ways of arriving at branch one under a single scalar margin rather than the full weighted objective; the new type keeps those three variants for compatibility and adds branches two and three as outcomes the existing implementation currently has no way to represent at all.

4 What the exercise of implementing surfaced

Two findings emerged in the course of writing the accompanying tests, neither of which was obvious from the prose specification alone, and both of which are properties of the objective rather than bugs in the code that computes it.

The first concerns δ_{dep} against an empty reference class. The Repair-Warrant Criterion requires $J(\emptyset \mid h) = 0$ for any history, and the natural way to check that requirement in code is to evaluate J at the null intervention and assert the result is zero. Doing so against an empty reference class fails, not because the criterion is wrong, but because Δ_{dep} becomes the difference of two infinities, and $\infty - \infty$ is not zero, it is undefined. This is not a case the specification needs to handle, because an empty reference class is exactly the condition that routes to branch two before J is ever evaluated; the criterion is a claim about J once a reference class exists, not a claim that survives the reference class being absent. But the fact that a direct implementation of the criterion silently produces a nonsensical value, rather than an error, in the one case where the criterion does not apply is worth stating plainly, because it means any future extension of this apparatus that evaluates J without first checking which branch of Ext_{Γ} obtained is at risk of producing a plausible-looking but meaningless number rather than a visible failure.

The second concerns the relative weighting of E and M . An early version of the worked example scored both terms as proportional to the size of the intervention’s effect, on the reasoning that a larger intervention should cost more (larger E) and also, all else equal, do more (larger M). Under any weighting where νM ’s coefficient exceeds E ’s, this construction has the consequence that the cost and marginal-value terms alone, before Δ_{dep} is even considered, favor applying some nonzero intervention regardless of its direction, because the two terms are not independent scorings of different properties but the same scalar multiplied by two different constants. The intended branch-one scenario, an intervention paying more than it recovers, accordingly failed to refuse: a token intervention in the wrong direction still came out warranted, because its cost was outweighed by a marginal-value term defined to move in lockstep with that same cost. The fix in the worked example was to construct a candidate whose Δ_{dep} is positive (structurally worse) rather than to retune the weights, which is the more general lesson: E and M must be scored independently enough that they can disagree, or the weighting parameters λ , μ , ν are not doing the dis-

criminating work the objective's form suggests they are. This is precisely the kind of failure mode that is invisible in the objective's algebraic statement and visible only once a concrete E and M are run against a case constructed to distinguish them.

5 Scope and limitations

The implementation takes no position on what a history, a continuation, or an intervention actually are beyond a feature vector, and the feature-vector semantics used throughout, including elementwise addition for intervention application and Euclidean distance for the structural metric, are placeholders chosen for their computational convenience rather than any claim about what structure the theory under test actually has. The independence of Γ_h 's provenance from h , which the specification requires, is not something the type system enforces or could enforce; it is a property of how a caller constructs a `ConstraintStructure` value, and the crate can no more guarantee it than a type system can guarantee that a function labeled `random` is actually random. Both limitations are intentional: the value of a reference implementation at this stage is in making the taxonomy's branch structure and the objective's algebraic form executable and testable, not in prematurely fixing the semantic content that any particular application of the theory will need to supply.